In-Network Pooling: Contribution-Aware Allocation Optimization for Computing Power Network in B5G/6G Era

Zheng Di, *Student Member, IEEE*, Tao Luo, Chao Qiu, *Member, IEEE*, Cheng Zhang, Zhutao Liu, *Member, IEEE*, Xiaofei Wang, *Senior Member, IEEE*, and Jing Jiang, *Member, IEEE*

***Abstract*—The coming B5G/6G will bring a huge amount of challenging applications with zettabytes of information. Computing power network (CPN) can provide a promising solution by accelerating the proliferation of computing power from a set of data centers to a multitude of network edges. However, in dealing with resource-hungry and real-time applications, most of the existing research doesn't make good use of idle computing and caching resources, and is barely possible to evaluate the contribution of the individual providing the resource. Therefore, we propose the in-network pooling framework, derived from a novel modified deep reinforcement learning (DRL) scheme, in which the dynamic resource pool (RP) is first modeled to make full use of the idle network resources, then the jointly computing and caching problem is formulated as the maximization of long-term system utility. Finally, Attention-based Proximal Policy Optimization (APPO) is employed to solve the problem. Particularly, the integrated attention mechanism reveals the evaluation of the different RPs' contributions to the learning process. Experimental results also show the priorities of the proposed algorithm and outperform the other existing algorithms.**

***Index Terms*—Attention mechanism, computing and caching, computing power network (CPN), proximal policy optimization (PPO), resource allocation.**

Manuscript received 4 January 2022; revised 5 June 2022; accepted 19 November 2022. Date of publication 29 November 2022; date of current version 25 April 2023. This work was supported in part by the National Science Foundation of China under Grant 62072332, in part by China NSFC (Youth) through under Grant 62002260, in part by the Open Fund for Key Laboratory of Dependable Service Computing in Cyber Physical Society, Chongqing University, under Grant CPSDSC202209, in part by the Open Fund for Shaanxi Key Laboratory of Information Communication Network and Security, Xi'an University of Posts Telecommunications, under Grant ICNS201901, and in part by the Open Research Fund from Guangdong Laboratory of Artificial Intelligence and Digital Economy (SZ), under Grant GML-KF-22-03. Recommended for acceptance by Dr. Zhi Zhou. *(Corresponding author: Chao Qiu.)*

Zheng Di and Chao Qiu are with the College of Intelligence and Computing, Tianjin University, Tianjin 300072, China, and also with the Key Laboratory of Dependable Service Computing in Cyber Physical Society Ministry of Education, Chongqing University, Chongqing 400044, China (e-mail: di_zheng@tju.edu.cn; chao.qiu@tju.edu.cn).

Tao Luo, Zhutao Liu, and Xiaofei Wang are with the College of Intelligence and Computing, Tianjin University, Tianjin 300072, China (e-mail: luo_tao@tju.edu.cn; liuzhutao@tju.edu.cn; xiaofeiwang@tju.edu.cn).

Cheng Zhang is with the Institute of Technology, Tianjin University of Finance and Economics, Tianjin 300222, China (e-mail: zccode@gmail.com).

Jing Jiang is with the Shanxi Key Laboratory of Information Communication Network and Security, Xi'an University of Posts and Telecommunications, Shaanxi 710061, China (e-mail: jiangjing@xupt.edu.cn).

Digital Object Identifier 10.1109/TNSE.2022.3225292

I. Introduction

In THE coming era, B5G/6G will support a decade of various massively data-intensive applications (e.g., virtual and augmented reality, brain communication interface). These applications generate zettabytes of digital services which require higher computation abilities, lower latency and wider coverage for a variety of network operations [1]. To meet the challenge of B5G/6G business, computing power network (CPN) is proposed to enable the deep integration of computing power and network architectures, accelerating the proliferation of computing capacity from a set of data centers to a multitude of network edges (e.g., base station (BS)) and terminal users (e.g., mobile devices) and endowing the infrastructures with computing and caching functionalities [2].

In-network pooling is one of the promising technologies of CPN in B5G/6G era, which connects local devices through the network orchestrations, and maps these ubiquitous and heterogeneous computing resources to a unified dimension to form resource pools (RPs) in the ad-hoc way. Generally, there are several RPs in one system and each of them can provide the resources to satisfy various demands.

Some of the studies investigate the solutions to construct RP in the manner of device-to-device (D2D) communications by organizing the scattered computing power of mobile users in B5G/6G networks, and orchestrated by the edge server to form the RP [3], or the edge server directly places nodes with computing capacity in the pool to perform tasks [4]. However, these solutions only consider the sharing of processing capacity among end devices to construct a RP, while ignoring the ability of edge servers and cloud servers, lacking effective utilization of resources.

A flexible approach to resource allocation becomes critical in B5G/6G era. Heterogeneous devices not only provide constantly changing resources, but also move with user mobility, it is necessary to solve the problem of resource scarcity and real-time application requirements in B5G/6G environments. Recently, learning-based algorithms are constantly being proposed to effectively implement resource issues in the cloud-edge-end architecture with more intelligent solutions [5], [6]. Particularly in the CPN, the interaction between various RPs and the users increases the randomness and complexity of the

environment. To tackle this problem, studies [7], [8], [9] apply deep reinforcement learning (DRL) to realize the tradeoff of the overall system utility by interacting with the environment through agents. When DRL perceives the external environment, all attention is equally placed on all RP allocation details, and the next strategy is determined by obtaining the state details of all RPs, which affects the decision-making efficiency of the agent. For CPN, especially in joint assignment optimization in B5G/6G networks, resources are abstracted into computable modules, which requires a system to quantify the contribution of each agent involved in learning. Based on the above discussion, there are some emerging issues that need to be addressed for the joint allocation optimization problem in CPN.

- **Inefficient Utilization:** There are large volumes of idle computing and caching resource, which is not made full use of. Particularly, with the users' mobility in B5G/6G, the joint optimization problem is more complicated.
- **Complicated Environment:** Existing work cannot satisfy the requirements for the environment-sensitive interaction between agents and various demands.
- **Low Performance:** The system is barely possible to evaluate the contribution of each agent participating in the learning, resulting in lower performance efficiency.

To address the aforementioned challenges, we jointly consider concentrating idle computing and caching resources in RP to enhance the performance of CPN. In this paper, we propose the architecture of in-network pooling and consider a 4-layer hierarchical architecture consisting of infrastructure, resource pooling, CPN dispatcher and application service. The architecture will be introduced in detail in Section III-A.

Typically, the agent needs to select actions through a higher level of perception, which improves the training efficiency and reduces the overhead. However, how the agent has a more efficient and higher-level perception is still a challenge. Based on this motivation, we introduce an attention mechanism into the DRL model. The attention mechanism model pays more attention to the RPs that contribute more to the system utility, while ignoring other RPs that have little effect on the objective function, enabling the agent to explicitly model the contribution of each RP to the system adaptively. Once the resource status of an RP is not good and the weight is reduced, the agent quickly adjusts the allocation strategy.

The main contributions are listed in the following:

- To achieve remarkable resource utilization and meet the computing power demand of digital transformation, we construct dynamic RPs in the CPN to make maximum use of idle resources in a cloud-edge-end hierarchical architecture.
- We use DRL to make computing and caching decisions between various RPs that are beneficial to the objective function in the face of complex and dynamic environmental conditions to maximize long-term system utility.
- We propose a novel Attention-based Proximal Policy Optimization Algorithm, named APPO, to deal with the high space and computation complexity and identify the contributions of different RPs to the system to better utilize the advantages of each RP. Experimental results also show the priorities of the proposed APPO algorithm.

The rest of the article is organized as follows: Section II surveys the existing work of CPN and introduces traditional and learning-based optimization methods for computing and caching. We specifically describe the goals of the system model in Section III. The problem formulation is introduced in Section IV. Section V shows the design of the proposed APPO algorithm. The performance evaluation is shown in Section VI. Finally, we conclude the article in Section VII.

II. Related Work

A. Computing Power Network

The CPN is mainly to integrate the resources provided by the end-edge-cloud infrastructure to meet the needs of B5G/6G networks. For this new type of network architecture, Wang et al. [10] created the framework to allow users of computing power to be more adaptable, network operators to be more flexible, and computing capality providers to be more profitable. Instead of assigning computing duties to network devices, Hu et al. [11] proposed an energy-efficient in-network computing paradigm for 6G that combines network operations into a general computing platform. Tang et al. [12] demonstrated that the CPN could effectively meet the flexible scheduling requirements of computing, storage and network resources for future 6G services. Besides, it can adapt to the computing power and network integration requirements in various scenarios. It can be seen that the new architecture of CPN integrates the network and computing and provides external computing services.

B. Resource Pooling

The resource composition of RP includes aggregation, amalgamation and resource shift [13]. After configuring resources, each RP sends its own resource specification and configuration to the CPN dispatcher, and the CPN dispatcher allocates RPs according to the task computing power requirements to ensure that the task requirements are met. It is worth noting that the resource size of the RP changes dynamically, and the changing environment is notified to the CPN dispatcher through the network. For efficient data exchange among resources and to maintain the QoS of applications, the choice of communication networks and protocols is crucial [14].

At present, there are studies that combine resources of the same type, such as aggregating storage resources in an end-to-end method to provide the system with greater storage capacity, or aggregating multiple computing resources of the same type. However, they do not consider the combination of storage and computing resources.

Different types of resources are brought together to meet the diverse needs of users. In addition, authors in [15] combined computing resources on the cloud to create RPs by virtualizing resources, and others [16], [17] consider hetorogeneous resources on the edge-cloud infrastructure to construct RPs. However, none of them consider merging computing in the end-edge-

cloud architecture in B5G/6G era. The same type and heterogeneous resources on the end-edge-cloud can be quantified into a unified dimension to provide customized services for users.

C. Traditional Resource Optimization

Traditional optimization methods like contract theory, game theory, and convex optimization had been widely used to optimize the problem of resource allocation. Authors in [18] paid their attention to the multivessel computation offloading problem and proposed an improved Hungarian Algorithm to solve the problem. Different from the existing work that only optimizes task offloading model [19], authors in [20], [21] discussed the problem of energy efficiency in edge computing networks in order to effectively use resources. While [22] discussed the resource optimization problem and modeled it into a joint stochastic mixed-integer nonlinear programming problem, then they used convex decomposition methods and matching game to solve it. In the Internet of Things (IoT) networks, the game method can also be employed to optimize the resource allocation [23], [24]. What's more, overall delay in system is also a focus of some researchers, authors in [25] formulated the computation task offloading problem into a distributed mirror prox optimization to deal with the unknown time-varying system cost.

However, most of the frameworks designed in existing research lack the learnability of dynamic environments, and although some of them work online scheduling, they only focus on the revenue of task completion rather than the network utility consisting of revenue and cost.

D. Learning-Based Resource Optimization

The increase of mobile devices, data and environment randomness have exacerbated the complexity of resource allocation. Thus, researchers proposed to use methods like Machine Learning, DRL to make complicated resource allocation decision. In the surveys [26], [27], [28], authors discussed the potentials of deploying a learning-based algorithm to solve the resource allocation. First, DRL algorithms can solve the complexity challenge brought by the changing environment [29], [30]. Second, learning-based method can providing guidance for intelligent resource management and balance the resources and workloads [31]. What's more, based on the universal model-free DRL framework, authors constructed a computing resource trading system with the continuous double action under the blockchain-based architecture [32]. Besides, learning algorithm can also handle the online task scheduling with a near-optimal performance in large-scale network [33].

Other methodologies like the Federated Learning allow the edge nodes to ensure privacy and security and efficient communication by storing data at the edge nodes to collaboratively learn and share models [34], some authors proposed the strategies of Federated Learning to solve the resource allocation and scheduling problem [35], [36], [37].

However, most of the authors considered the interaction between the system and the environment, but the contribution ability of each agent involved in the learning process was not fully paid attention to.

TABLE I
SUMMARY OF IMPORTANT NOTATIONS

Notations	Definition
$\mathcal{M}$	Set of RPs or BSs
$\mathcal{N}$	Set of mobile devices
$\mathcal{T}$	Set of time slots
$\mathcal{P}(t)$	Set of computing power queue lengths of all RPs
$P_m(t)$	Computing power queue length of RP m
$p_m(t)$	Total available harvested computing power
$\eta_m(t)$	Harvested computing power in the RP m
$\eta_m^a(t)$	Harvested computing power due to completion of tasks in the RP m
$\eta_m^o(t)$	Harvested computing power by the user entering the RP m
χ_m	Consumed computing power in the RP m
χ_m^a	Consumed computing power due to the execution of tasks in the RP m
χ_m^o	Consumed computing power by the user leaving the RP m
$\mathcal{K}$	Set of tasks
O_k	Data size of task k
ϕ_k	Required CPU cycles of task k
ω_k	Revenue for processing the computation task k
$\mathcal{J}_{m,k}^p$	Set of whether the RP m perform task k
$\mathcal{J}_{m,c}^e$	Set of whether the RP m cache data c
$U(t)$	Utility of the system
$U^p(t)$	Utility of task offloading
$U^e(t)$	Utility of data caching
$\mathcal{S}, \mathcal{A}, R$	The state space, action space, and reward, respectively

III. SYSTEM MODEL

A. Network Architecture

We consider that there are multiple cellular networks located by BSs in the scene. Each cellular network represents one RP. Therefore, each RP includes one BS and several mobile devices under its coverage. Both of them can provide *computing-* and *caching-* resources according to their own conditions. In the CPN, we define RP M represented by the BS as $\mathcal{M} = \{1, 2, \ldots, M\}$. Since one BS represents one RP, for simplicity, we also use M to denote the BS. And $\mathcal{N} = \{1, 2, \ldots, N\}$ mobile devices are in the coverage of BSs. The system operates over unit-size time slots denoted by $\mathcal{T} = \{1, 2, \ldots, T\}$, $t \in \mathcal{T}$. The mobile device $n \in \mathcal{N}$ communicates with BS through the wireless link. The important notations are listed in Table I.

Fig. 1 shows the 4-layer architecture we proposed, the infrastructure layer includes end nodes, edge nodes, and cloud servers. They all provide computing and caching resources in the CPN network. According to the previous modeling of RP,

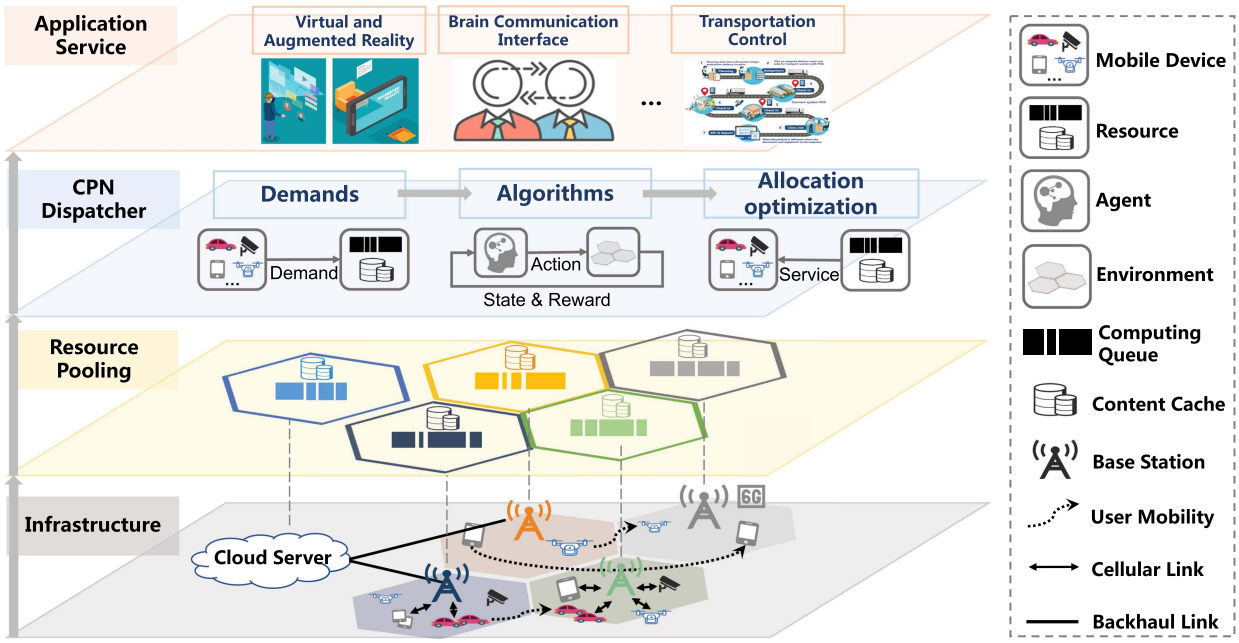

Fig. 1. Architecture of in-network pooling for CPN in B5G/6G era.

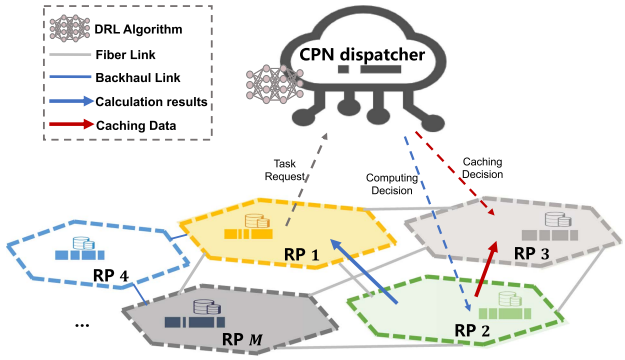

Fig. 2. System model.

resources are pooled and idle resources are concentrated to form individual RPs in the resource pooling layer. The third layer is the CPN dispatcher layer. The functions of this layer include acquiring users' demands. According to the demands, the system resources are allocated by the proposed learning algorithm. In addition, there is a resource optimization function to achieve the final effect. The top layer is the application service layer used to serve users. Its application services include virtual and augmented reality, brain communication interface and some system services for monitoring and control, such as traffic control, data monitoring, etc. Let's take a use case of our architecture and Fig. 2 shows the system model. The user puts the meaningful datasets in the Multiple Object Tracking Benchmark (MOT) datasets on the cloud server in RP 4 so that all users can do video processing services in the future. The MOT datasets focus on pedestrian tracking [38], and when the user wants to do video tasks, they will retrieve a part of the data from the cloud for analysis. Specifically, when a user in RP 1 wants to do a video tracking task, the task request is first sent to the CPN dispatcher in our architecture. The dispatcher obtains the final resource allocation strategy after passing the algorithm proposed by us according to user requirements, including which RP computes the computing task and where the required data is cached. If the dispatcher decides the task to be computed in RP 2, it will send a message to the BS in RP 2, and the BS will prepare corresponding computing resources after receiving the message to perform the task calculation. If the dispatcher decides to buffer the data in RP 3, RP 3 will first check whether the neighbor PR has the data, and if so, it will directly buffer the corresponding data of the neighbor RP, if not, it will download it from the cloud server in RP 4.

B. Dynamic Resource Pool Model

Let $\mathcal{P}(t) = \{P_1(t), \ldots, P_m(t)\}$ denote the set of computing power queue lengths of all RPs, and $P_m(t)$ denotes the computing power queue length of RP m. To model the dynamic nature of the computing harvested process, we consider the tasks and devices' mobility. Specifically, as mobile devices enter and leave the RP, the computing power of the corresponding RP will increase and decrease. Similarly, the execution and completion of tasks will also bring about changes in computing power. At time slot t, the harvested computing power by tasks execution and users mobility in the RP m is $\eta_m(t) = \eta_m^a(t) + \eta_m^o(t)$, and consumed computing power is $\chi_m(t) = \chi_m^a(t) + \chi_m^o(t)$. The following will specifically introduce the task model and mobility model so as to better build the dynamic RP queue model.

1) Task Execution Model: We model the task arrival rate as a Poisson process with a rate $\lambda_{m,t}^K$, indicating the expected number of computation tasks arriving in the task queue in each time slot. The available calculation task at time slot t is identified as $\mathcal{K}(t)$ with $K(t) = \|\mathcal{K}(t)\|$. A 3-tuple parameter $< O_k(t), \phi_k(t), \omega_k(t) >$ can be used to label the

characteristics of the computation tasks, where k indicates the index of a computation task, the data size of task k is denoted by $O_k(t)$, the required CPU cycles are denoted by $\phi_k(t)$, and the revenue for processing the computation task is $\omega_k(t)$. We assume the computation time of task k cannot exceed the length of one time slot. Let $\mathcal{J}^p(t) = \{J^p_{m,k}(t) \,|\, m \in \mathcal{M}, k \in \mathcal{K}\}$ denote whether the task k is executed by the RP m. Denote $J^p_{m,k}(t) = 1$ if RP m executes task k in time slot t, otherwise $J^p_{m,k}(t) = 0$. Each task requires only one RP for execution, can be expressed as

$$\sum_{m \in \mathcal{M}} J^p_{m,k}(t) \leqslant 1 \tag{1}$$

When the task is executed at $t-1$, this part of the computing power will be released at time t, and the RP will reuse this part of the computing power, in this way, the computing power collected at t due to the completion of tasks is expressed as

$$\eta^a_m(t) = \sum_{k \in \mathcal{K}(t-1)} \phi_k J^p_{m,k}(t-1) \tag{2}$$

The computing power consumed by performing tasks in the t-th time slot is

$$\chi^a_m(t) = \sum_{k \in \mathcal{K}(t)} \phi_k J^p_{m,k}(t) \tag{3}$$

2) Mobility Model: We use a Hidden Markov Model (HMM) [39] to predict the probability of mobile users n entering each RP. Specifically, the HMM applies a forward-backward algorithm to predict the location of users in the near future based on the observed mobility trace. The position observation sequence of user n is denoted as $\mathcal{L}_n = \{l_1, l_2, .., l_t, \ldots\}$, where l_t is the position of the user n at time slot t. According to the user's location l_t, we can know that the sequence of entering the RP is $\mathcal{Z} = \{z_1, z_2, \ldots, z_t, \ldots\}$, $z_t \in \mathcal{M}$.

First, the model λ^H of HMM can be expressed by a 3-tuple parameter $< g, A, B >$, where $g = \{g_i\}$ denotes the set of initial hidden state probabilities, $\mathbb{A} = \{A_{ij}\}$ is the set of the transition probabilities from the hidden states $\mathbf{\Omega}_i$ to $\mathbf{\Omega}_j$, and $\mathbb{B} = \{B_i(\tau)\}$ indicates the set of probabilities of the observable state σ_t at time τ in the hidden state $\mathbf{\Omega}_i$, the g_i, A_{ij} and $B_i(\tau)$ can be calculated as follow

$$\begin{cases} \bar{g}_i = \varrho_t(i) \\ \bar{A}_{ij} = \dfrac{\sum_{t=1}^{T} \psi_t(i,j)}{\sum_{t=1}^{T} \varrho_t(i)} \\ \bar{B}_j(\tau) = \dfrac{\sum_{t=1,\sigma_\tau}^{T} \varrho_t(j)}{\sum_{t=1}^{T} \varrho_t(j)} \end{cases} \tag{4}$$

where $\bar{g}_i$ represents the probability that the user stays at the hidden state $\mathbf{\Omega}_i$ at t, $\bar{A}_{ij}$ is the probability of transition from $\mathbf{\Omega}_i$ to $\mathbf{\Omega}_j$, and $\bar{B}_j(\tau)$ indicates the probability of observing the state o_c when the hidden state is $\mathbf{\Omega}_j$. Given the model λ^H and observation sequence, $\varrho_t(i) = P(s_i|\sigma_1\sigma_2 \ldots \sigma_T, \lambda^H)$ is the probability of state $\mathbf{\Omega}_i$ at time t, $\psi_t(i,j) = P(\mathbf{\Omega}_i, \mathbf{\Omega}_j|\sigma_1\sigma_2 \ldots \sigma_T, \lambda^H)$ is the probability of transition from $\mathbf{\Omega}_i$ to $\mathbf{\Omega}_j$ at time t.

Then, combine the formula (4) and the observed state σ_τ which is the position l_τ to predict the probability of the user entering every RP at time $\tau + 1$. We choose the one with the highest probability $p_{i \to j}(t)$ as the pool that the mobile users enter from RP i to RP j at the beginning of t.

Finally, let h_n denote the idle computing power that mobile device n can contribute to the RP. The computing power harvested by devices entering the RP is

$$\eta^o_m(t) = \sum_{n \in \mathcal{N}} h_n p_{j \to m}(t) \tag{5}$$

The computing power consumed by devices leaving the RP is

$$\chi^o_m(t) = \sum_{n \in \mathcal{N}} h_n p_{m \to i}(t) \tag{6}$$

To provide a better service, the RPs aim to gather as much computing power as possible. However, due to the limited spectrum resources of the BS, it is impossible to serve too many users who can provide computing power, so that the computing power resources that can be provided to the RP are reduced, so we set the upper bound Λ of the computing power of the RP, and the computing power queue length $P_m(t)$ cannot higher than capacity Λ [40].

$$P_m(t) \leq \Lambda, m \in M \tag{7}$$

We denote the harvested computing power $p_m(t)$ of RP m at time slot t as

$$p_m(t) = min(\Lambda - P_m(t), \eta_m(t)) \tag{8}$$

The dynamics of RP's computating power queue across time slots can be described as

$$P_m(t+1) = P_m(t) - \chi_m(t) + p_m(t) \tag{9}$$

The constraint of the consumed computing power cannot exceed the available power, shown as

$$\sum_{k \in \mathcal{K}(t)} \phi_k J^p_{m,k}(t) \leqslant P_m(t) \tag{10}$$

C. Computing Model

We denote the computation capability allocation ratio of RP m allocated to task k as v^k_m, its value range is 0 to 1. In the computing power network, the more free computing power in the RP, the greater the potential computing power services it provides. However, because not all free computing power is provided to one task, and different tasks have different characteristics, the more dispersed computing power cannot support a large task, the quality of computing power that can be provided by the RP for tasks is different. The above two points result in the fact that we are unable to determine how much computing power can be provided for different tasks at the next time slot. Therefore, the computing power supply capacity ratio v^k_m can be

simulated as a random variable and separated into discrete levels denoted by $\delta = \{\delta_0, \delta_1, \ldots, \delta_{G-1}\}$ where G is the number of states. Assume that the computation capability allocation ratio realization of $v_m^k(t)$ to be $\Theta_m^k(t)$ at time slot t. The transition of the computation capability allocation ratio of pool m for task k can be modeled as a Markov chain [41]. We defined the transition probability as $d^p_{x_k y_k}(t) = Pr(\Theta_m^k(t+1) = y_k | \Theta_m^k(t) = x_k)$, and $x_k, y_k \in \delta$. The $L \times L$ computation ratio state transition probability matrix is defined as

$$\zeta_m^k(t) = \left[d^p_{x_k y_k}(t) \right]_{L \times L} \tag{11}$$

D. Caching Model

The set of data in the CPN system is denoted by $\mathcal{C} = \{1, 2, \ldots, C\}$, the requested data is denoted as c, $c \in \mathcal{C}$. Similar to [42], the average request rate for data c at time t can be denoted as

$$\lambda_c^Q(t) = \frac{\lambda^E}{\rho c^\beta} \tag{12}$$

We assume that the probability of requesting data is determined by a Zipf-like distribution, and denote β as the Zipf slope, the value of β ranges from 0 to 1. Thus the probability of data c being selected is $\frac{1}{\rho c^\beta}$, where $\rho = \sum_{c=1}^{C} \frac{1}{c^\beta}$. We model the request arrival rate for data C as a Poisson process with rate λ^E. Let $\mathcal{J}^e(t) = \{ J^e_{m,c}(t) \,|\, m \in \mathcal{M}, c \in \mathcal{C} \}$ denote whether the data c is cached by the RP m. Denote $J^e_{m,c}(t) = 1$ if RP m executes data c in time slot t, otherwise $J^e_{m,c}(t) = 0$.

The random variable ξ_c can be used to determine whether or not an RP's requested data c is in the cache. State 1 means the data c is in the cache, state 0 means not. We model the state of the cache as a two-state Markov chain and state space can be denoted as $\mathcal{W} = \{0, 1\}$, where $\xi_c \in \mathcal{W}$. Thus the transition probability is $d^E_{x'_c y'_c}(t) = Pr(\xi_c(t+1) = x'_c \,|\, \xi_c(t) = y'_c)$, and x'_c, $y'_c \in \mathcal{W}$. The cache state transition probability matrix of data c is defined as

$$\Gamma_c(t) = [d^E_{x'_c y'_c}(t)]_{2 \times 2} \tag{13}$$

Users in the CPN system are assumed to have different computing power. To simplify the spectrum utilization between users and BSs, we introduce a virtual node $u_0(t)$ to represent the total user status in the m RP at time slot t. Specifically, users have location coordinates and different computing power levels, that is, each node in the RP has different weights. Using the method of taking the center of gravity node, the nodes are used as vertices, and connected to form a polygon, and the polygon is divided into multiple triangles, and the center of gravity of each triangle with weight to get the center of gravity of the polygon.

We denote the spectrum bandwidth orthogonally assigned from BS m to users that are covered in the mth RP as $b_{u_0,m}$, thus, user n receives no interference from other users connected to BS m. The achievable spectrum efficiency of user n linked with BS m can be expressed as $w_{u_0,m}$ and it can be easily achieved using Shannon bound.

IV. Problem Formulation

In this article, our goal is to maximize the long-term utility of the system $U(t) = U^p(t) + U^e(t)$, $U^p(t)$ is the utility of task offloading and it can be expressed as

$$U^p(t) = \sum_{m=1}^{M} \sum_{k=1}^{K} U^p_{m,k}(t) \tag{14}$$

where the energy consumption of m pool for performing one CPU cycle is denoted as e_m W/GHz, and the price per Joule for hardware energy consumption is denoted as f_m. The utility $U^p_{m,k}(t)$ of task k to RP m can be calculated as

$$\begin{aligned} U^p_{m,k}(t) = J^p_{m,k}(t)(\Theta^k_m(t) P_m(t) \frac{O_k(t)}{\phi_k(t)} \omega_k(t) \\ - \phi_k(t) e_m f_m) \end{aligned} \tag{15}$$

The caching utility $U^e(t)$ can be expressed as

$$U^e(t) = \sum_{m=1}^{M} \sum_{c=1}^{C} U^e_{m,c}(t) \tag{16}$$

where the backhaul cost which can be saved when the data is cached is denoted as κ_{u_0}, ϑ_m is the size of the data to be cached and the spending of the space occupied by the cache is denoted as φ_m. The utility $U^e_{m,c}(t)$ of data c to RP m can be calculated as

$$U^e_{m,c}(t) = J^e_{m,c}(t)(\kappa_{u_0} b_{u_0,m} w_{u_0,m} \Xi^c_m(t) - \vartheta_m \varphi_m)) \tag{17}$$

Thus, we formulate the optimization problem to maximize the long-time system utility.

$$\begin{aligned} \max_{J^p_{m,k}(t), J^e_{m,c}(t)} \quad & \lim_{T \to \infty} \frac{1}{T} \sum_{t \in \mathcal{T}} \mathbb{E}[U](t) \\ \text{s.t.} \quad & (1), (7), (8), (10) \end{aligned} \tag{18}$$

V. Attention-Based Ppo Framework

A. DRL Model Design

1) System State: The state vector can be described as

$$\begin{aligned} \mathcal{S}(t) = [& \Theta^1_1(t), \Theta^2_1(t), \ldots, \Theta^K_1(t), \\ & \Theta^1_2(t), \Theta^2_2(t), \ldots, \Theta^K_2(t), \ldots, \\ & \Theta^1_m(t), \Theta^2_m(t), \ldots, \Theta^K_m(t), \\ & \Xi^1_1(t), \Xi^2_1(t), \ldots, \Xi^C_1(t), \\ & \Xi^1_2(t), \Xi^2_2(t), \ldots, \Xi^C_2(t), \ldots, \\ & \Xi^1_m(t), \Xi^2_m(t), \ldots, \Xi^C_m(t)] \end{aligned} \tag{19}$$

The state $\Theta^K_m(t)$ indicates the effective computing power weight that RP m can allocate to task k. The state Ξ^C_m indicates whether or not the RP requested data in the cache. Here, $\Xi^C_m(t) = [\xi_1(t), \xi_2(t), \ldots, \xi_I(t)]$, for $\forall i \in \{1, 2, \ldots, I\}$, $\xi_i \in \{0, 1\}$.

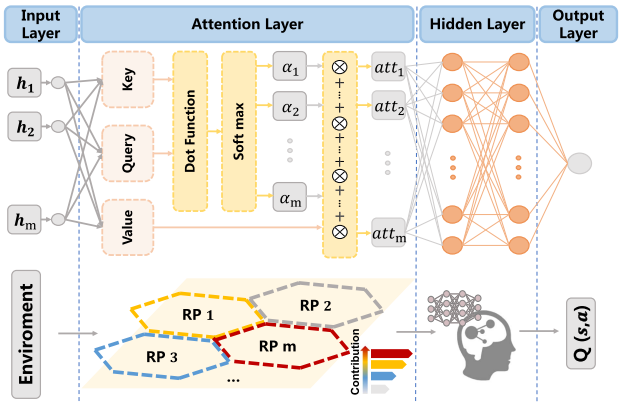

Fig. 3. Attention-based Actor-Critic.

2) System Action: The action vector can be described as

$$\mathcal{A}(t) = \{a_1^p(t), a_1^e(t), a_2^p(t), a_2^e(t), \ldots, a_m^p(t), a_m^e(t)\} \tag{20}$$

where $a_m^p(t) = [a_{m,1}^p(t), a_{m,2}^p(t), \ldots, a_{m,K}^p(t)]$, the action $a_{m,k}^p(t)$ represents the whether the RP m performs task k at time slot t, where $a_{m,k}^p(t) = 1$ means the RP m performs task k, $a_{m,k}^p(t) = 0$ means not. Also, $a_m^e(t) = [a_{m,1}^e(t), a_{m,2}^e(t), \ldots, a_{m,C}^e(t)]$, where $a_{m,c}^e(t)$ represents whether to cache data c for RP m at time slot t, where $a_{m,c}^p(t) = 1$ means the RP m caches data c, $a_{m,c}^p(t) = 0$ means not.

3) Reward Function: Reward represents the value brought by an action. In this article, we defined that the reward R obtained from the offloading decision and caching decision made by each time slot t. Based on the system model, we define the reward as the system utility that contains both computing and caching. We denote reward as $R(t) = U(t)$.

We propose an attention mechanism based Proximal Policy Optimization (PPO) [43] algorithm, named APPO, which is a modified actor-critic framework to solve the optimization problem. The APPO framework includes three neural networks, the old and new actor neural networks and the critical neural networks. The actor network is to select actions, and the critical network is to calculate the value of the corresponding actor to evaluate the quality of actor training. Specifically, Actor neural network we used is introduced in Section V-A4 and the attention mechanism [44] is added to the critical neural network, which is introduced in Section V-A5.

4) Actor Network: To improve the training speed and make the sampling data reusable, the actor neural network includes a new actor network and an old actor network. The similarity of action probability between the two actors is used to limit the update range of the new strategy. The network architecture of the old and new actors is the same. The actor network takes the observation state of the local RP as the input layer, and it includes one hidden layer, which adopts the $Relu$ activate function. The $Relu$ function is a non-linear activation function in the neural network. A softmax layer outputs the probability of all actions.

5) Critic Network With Attention Mechanism: The Critic network is similar to the actor network. We add the attention layer after the input layer. Since the mobility of the user changes the computing power and storage resource size in the RP, and the user moves between adjacent BSs, the complex environment makes it difficult to focus our limited efforts on effective RPs. Therefore, we introduce an attention mechanism into the algorithm model, add the attention mechanism layer to the critic network. This layer perceives the impact of the potential RP characteristics on the system utility by inputting the states of each RP to record the weight of RP in the system and to coordinate and balance from a variety of perspectives. The attention layer shown in Fig. 3 is aware of the contribution of each RP to the system. The red pool indicates that the RP weight is high, which means that the contribution of the pool is large, so the neural network pays more attention to the RP, and the gray pool indicates that its contribution is small.

Using the query-key-value idea of attention mechanism and finding the relationship between pools through the weight matrix, we first define h_i as the state of the ith RP, that is, the system state χ_i defined in Section V-A1. Then, in order to improve the fitting ability of the model, matrices W_q, W_k, and W_v is introduced to make a linear change to h_i, and these W matrices can be trained. The query q_i, the key k_i, and the value v_i can be calculated as follow:

$$q_i = W_q h_i, k_i = W_k h_i, v_i = W_v h_i, \forall i \in \mathcal{M} \tag{21}$$

In order to obtain the similarity scr_{ij} between q_i element of RP i and k_j element of RP j, we apply alignment function (e.g., dot product) D to score how relevant between states, $scr_{ij} = D(q_i, k_j), \forall i, j \in \mathcal{M}$. Alignment function scores are mapped to attention weights using distribution functions $softmax$. The attention score α_{ij} can be calculated as follow

$$\alpha_{ij} = softmax(scr_{ij}), \forall i, j \in \mathcal{M} \tag{22}$$

Finally, in order to retain the input features the model introduced the value v_i, and the attention value att_i is obtained as follow

$$att_i = \sum_{i=1}^{M} \alpha_{ij} v_i = \sum_{i=1}^{M} \frac{e^{scr_{ij}}}{\sum_{i=1}^{M} e^{scr_{ij}} v_i}, \forall i, j \in \mathcal{M} \tag{23}$$

B. Algorithm Design

The algorithm design is shown in Fig. 4. In the first episode, the policy π_θ is initialized, and the agent uses the policy to interact with the environment for sampling and storing the data in a buffer, which includes the states s_t of RPs, the action a_t and the reward r_t collected by each step. After obtaining a complete batch of data, the actor network and the critic network learn from the data. The critic network obtained T estimated value functions during the learning process and sumed the discounted reward which is stored in the batch. Here, the advantage function used by the critic to evaluate the quality of the action a selected in the state s is as follows

Algorithm 1: APPO Algorithm

```
Initialize new actor network π_{θ_n} and critic network π_{θ_c}
for episode = 0, 1,...,N do
    Receive initial observation state s_0
    for t = 0,1,2,...,T do
        Select action a_t by running new actor with policy π_{θ_n}
        observe the reward r_t and next state s_{t+1} by selecting action a_t
        Collect the data {s_t, a_t, r_t} in the buffer ℝ
        Compute the ratio r_t(θ)
    end for
    Compute the advantage F̂_t by (24)
    π_{θ_old} ← π_{θ_new}
    Update critic network by (25)
    Update new actor network by (26)
end for
```

$$\hat{F}_\pi(s_t, a_t) = Q_\pi(s_t, a_t) - V_\pi(s_t) \tag{24}$$

where $Q_\pi(s_t, a_t)$ is the sum of discounted reward and can be expressed as $Q_\pi(s_t, a_t) = r_t + \gamma V_\pi(s_{t+1})$ according to the TD-error. The state value $V_\pi(s_{t+1})$ obtains through critic network. The value of the discount factor γ ranges from 0 to 1. Critic network used the mean-square function to update the critic network parameters, and the loss function is as follows

$$L^{MSE} = \mathbb{E}_{\pi_\theta}[V(t) - \hat{R}(t)]^2 \tag{25}$$

After the new actor completed a batch sampling operation, the strategy parameters are copied to the old actor. The PPO algorithm uses the importance sampling method to update the target strategy, and the importance weight is expressed as $r_t(\theta) = \frac{\pi_\theta(a_t, s_t)}{\pi_{\theta_{old}}(a_t, s_t)}$. Actor network uses the method of clipping parameters and TD-error to limit the change of each network update, and the loss function can be defined as

$$L^{CLIP} = \mathbb{E}_{\pi_{\theta_{old}}}[min((r_t(\theta))\hat{F}_t, g(\epsilon, \hat{F}_t))] \tag{26}$$

$$g(\epsilon, \hat{F}_t) = \begin{cases} (1+\epsilon)\hat{F}_t, & if\ \hat{F}_t \geq 0 \\ (1-\epsilon)\hat{F}_t, & if\ \hat{F}_t < 0 \end{cases} \tag{27}$$

VI. Performance Evaluation

In this section, we carry out many simulations to evaluate the performance of the proposed APPO algorithm. First, we present simulation settings, and then we present the simulation results to verify the strength of APPO algorithm in terms of computing and caching.

A. Simulation Settings

In our simulations, we use an Intel(R) Core(TM) i5 macOS Catalina 64-bit system to implement all the simulation experiments. We assume that there are four RPs and ten users in MEC environment, and users are spread randomly within each RP's covered region. The weight Θ_m^k of each pool for different tasks follows the Markov model. We divide the computing power quality of each pool into four levels, very low, low, medium and high. The transition probability matrix is as follows:

TABLE II
Main Simulation Parameters

Parameters	Value
o_k	1 Mbits
ϕ_k	5 Mcycles
ω_k	1 units/Mbps
k_{u_0}	3 units/Mbps
$b_{u_o,m}$	5MHz
$v_{u_o,m}$	[1, 3] bps/Hz
f_m	10 units/J
e_m	1 W/GHz
φ_m	3 units/Mbits
ϑ_m	1 Mbits
Θ_m^K	[0.01, 0.1, 0.5, 1]
Ξ_m^C	[0, 1]

$$prob_v = \begin{bmatrix} 0.5 & 0.3 & 0.15 & 0.05 \\ 0.3 & 0.5 & 0.05 & 0.15 \\ 0.15 & 0.05 & 0.5 & 0.3 \\ 0.05 & 0.15 & 0.3 & 0.5 \end{bmatrix} \tag{28}$$

Similar to the work in [42], the cache state follows the Markov model as well, with a transition probability of 0.4 from one state to the next and a transition probability of 0.6 from one state to the next, the other settings of parameters of the simulations are given in Table II.

In the neural network constructed in this article, the actor network consists of an input layer, a hidden layer and an output layer. Among them, the hidden layer is a fully-connected layer with 200 hidden nodes. In order to find the degree of influence of the service RPs on their own decision-making and make a judgment that is beneficial to the overall system. The critic network consists of an input layer, two hidden layers and an output layer. The hidden layer consists of an attention layer and a fully connected layer with 100 hidden nodes. The discount factor of APPO algorithm is 0.9.

We use five comparison algorithms to demonstrate the performance of APPO proposed in this article:

- *Actor-critic mechanism (AC) [45]:* AC is a reinforcement learning algorithm based on value and policy gradient.
- *Actor-critic with attention mechanism (AAC) [46]:* AAC is an actor-critic algorithm based on an attention mechanism.
- *Deep Q Network (DQN) [47]:* DQN is a reinforcement learning algorithm that is the combination of Q-learning and convolutional neural network (CNN).
- *Static Allocation [48]:* Each RP allocates computing power and cache resource quantitatively in turn.
- *Random Allocation [49]:* Computing tasks are randomly assigned to a certain RP, and data is also randomly cached in the RP in the system.

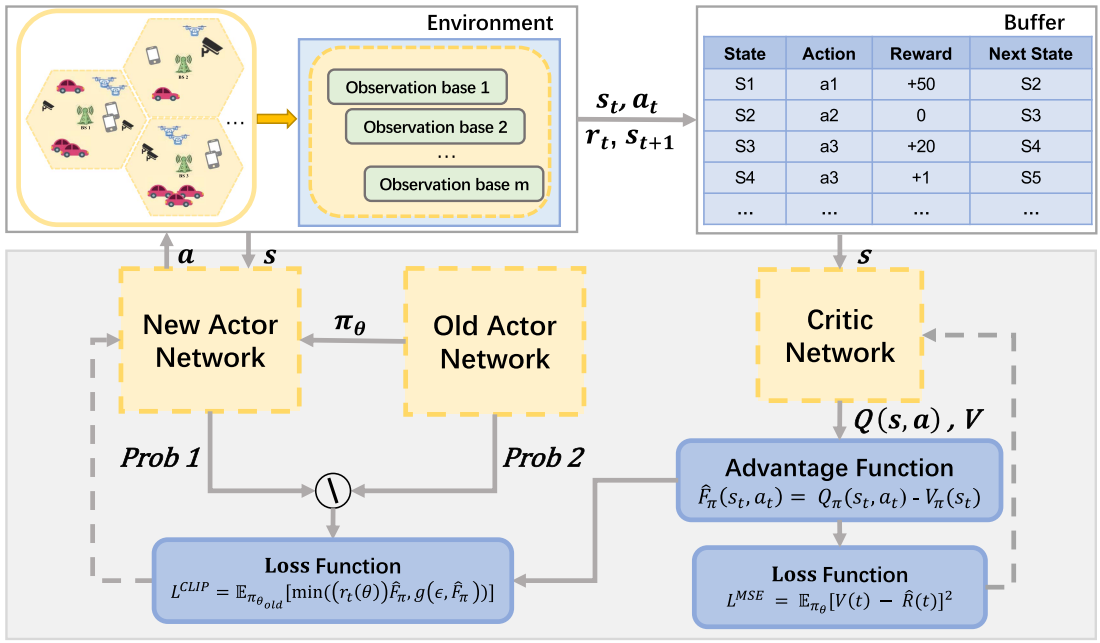

Fig. 4. Algorithm design.

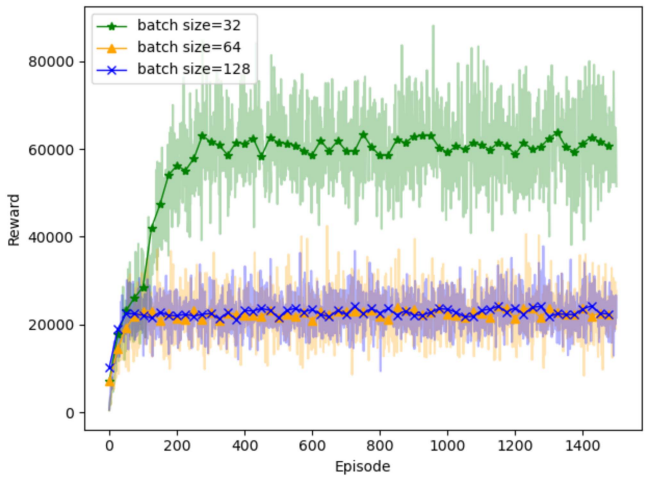

Fig. 5. Reward with different batch size.

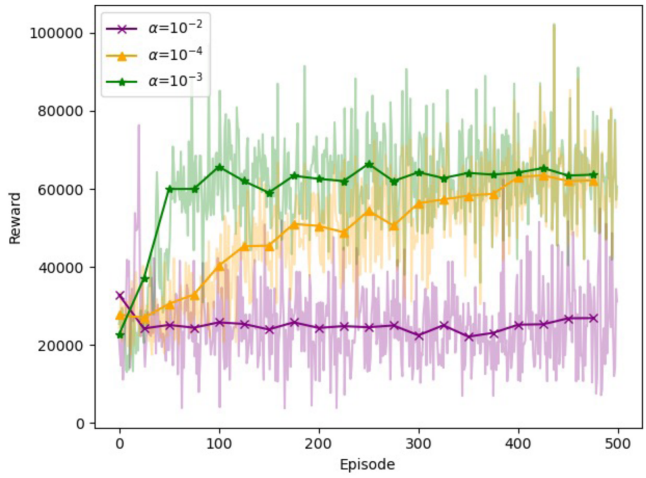

Fig. 6. Reward with different actor's learning rate.

B. Simulation Results

1) Comparison of Different Algorithm Parameters: Minibatch size is a very important hyperparameter for DNN. Fig. 5 shows the impact of minibatch size on the training process for APPO algorithms. We used 32, 64 and 128 minibatch sizes to train the proposed APPO respectively. From Fig. 5 we can observe that $minibatch\ size = 32$ gets the highest reward. There is not much difference between $minibatch\ size = 64$ and $minibatch\ size = 128$, and can quickly converge to a stable state, but the realized reward is not high. The reason is that too large minibatch size leads to too fast convergence and fewer training times so that the correction of the parameters is slower, the network can easily converge to some bad local optimal points.

Fig. 6 shows the effect of the actor's learning rate on the APPO proposed in this paper. When the learning rate $a = 0.001$, the reward is the highest, reaching about 60,000. It can

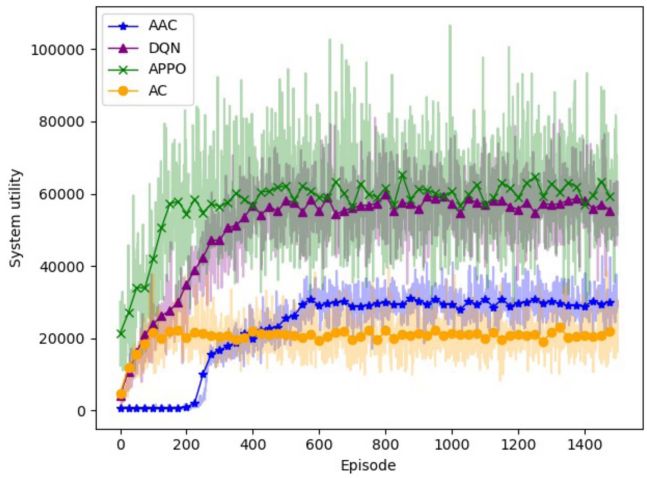

Fig. 7. System utility with different algorithm.

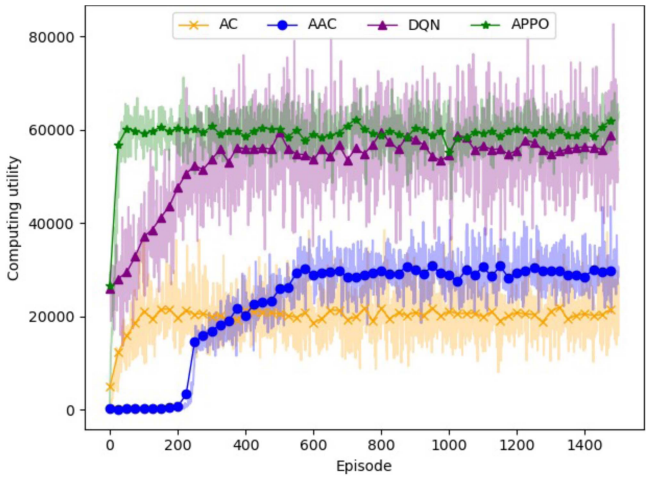

Fig. 8. Computing utility with different algorithm.

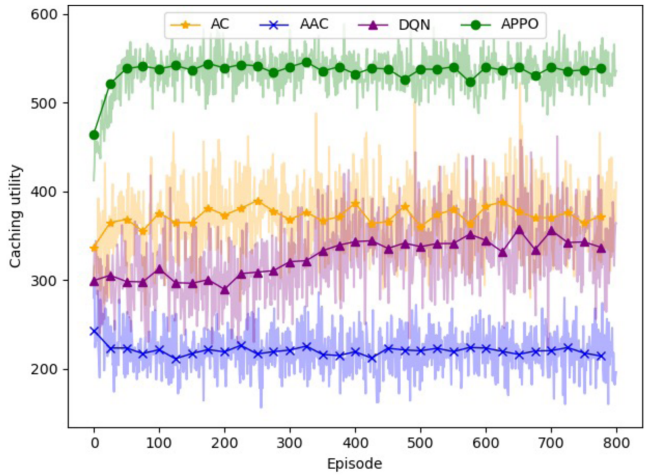

Fig. 9. Caching utility with different algorithm.

be seen from the figure that as the learning rate increases, the convergence speed becomes faster. Although the high learning rate can converge rapidly, when the learning rate $\alpha = 0.01$, it cannot get the high rewards brought by other learning rates, because the high learning rate misses the global optimization of the learning process, it will cause the network to converge to a local optimum and obtain a low utility.

2) Comparison of Different Algorithms: Fig. 7 shows the change curve of system utility of current episode to complete offloading and caching in different algorithms. Compared with other algorithms, the system utility achieved by the APPO algorithm gradually increases as the episode increases. It starts to converge after the 180 episodes and then fluctuates slightly. The reason is that when poor samples are selected, they may eventually reach the local maximum with a low reward value. APPO algorithm achieves the highest system utility including computing and caching, reaching about 60,000. The system utility implemented by DQN is slightly lower than the APPO algorithm, and AC gets the lowest system utility. On the one hand, the reason is that AC does not obtain the weight of RP to influence system utility, and cannot consider the impact of RPs on system utility from a global perspective. On the other hand, AC deals with discrete action spaces ranging from several dimensions to hundreds of dimensions, the effectiveness of the sampling is worse than that of DQN. Although DQN has advantages in dealing with this problem, the system utility obtained by APPO is still higher than DQN and faster than DQN. The reason is that APPO not only pays more attention to the state of RPs but also accelerates the learning process by adopting correct behaviors during training, which is better To obtain higher system utility.

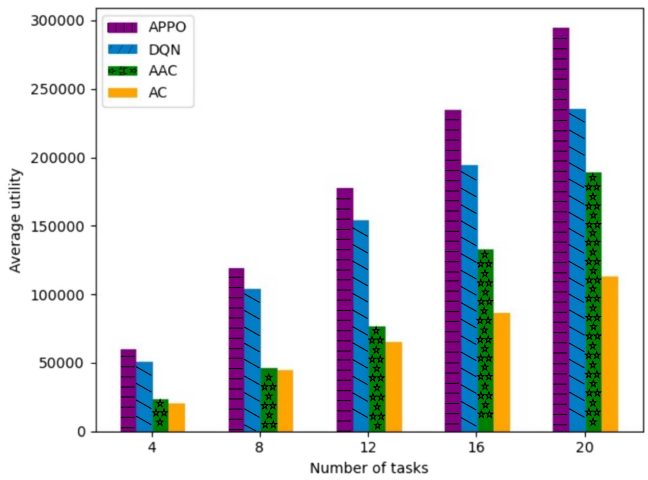

Fig. 10. The impact of the number of tasks on utility under different algorithm.

Fig. 8 shows the effect of different algorithms in realizing the computational utility of the system. The APPO proposed in this paper has the highest computational utility, converging at the 80th episode, and the DQN algorithm starts to converge at 300 episodes. The computational utility of DQN is about 55,000, which is about 5,000 lower than APPO. The AAC algorithm starts to converge to about 30,000 after 600 episodes, and its convergence speed is about 600 episodes slower than APPO. The reason is that the APPO algorithm has new and old networks to limit the network update range, and it uses off-policy to reuse sampled data which can improve the algorithm training speed.

Fig. 9 shows the effect of different algorithms in achieving system cache utility. The APPO proposed in this paper has the highest caching utility, starting to converge to 540 after 50 episodes with slight fluctuations. The AAC algorithm achieves the lowest utility. With the increase of episodes, the caching utility achieved by AAC drops from 250 to about 220, and it starts to converge after 40 episodes. The reason is that the probability of good samples when AAC explores the caching part of the system is too small, resulting in it can only converge to the local area. The experimental results prove that APPO can not only achieve higher system utility but also realize the highest caching utility included in the system utility more intelligently.

3) Comparision of Different System Parameters: Fig. 10 shows the effect of different numbers of tasks on the average utility of the system. It can be seen from Fig. 10 that the average utility of the system is on the rise as the number of tasks increases. Compared with the other three algorithms, the

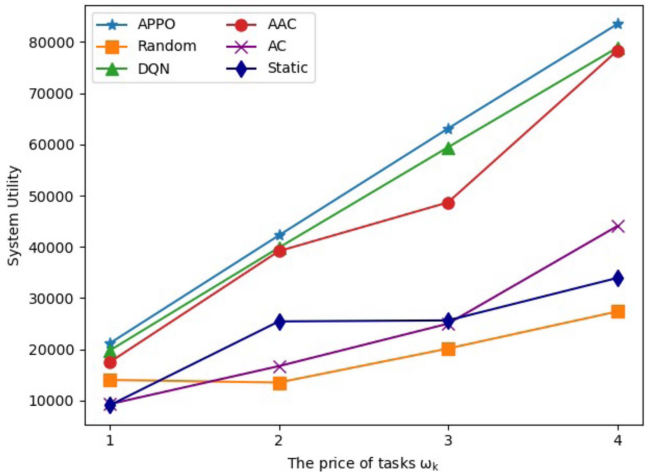

Fig. 11. The impact of the price of tasks on system utility under different algorithm.

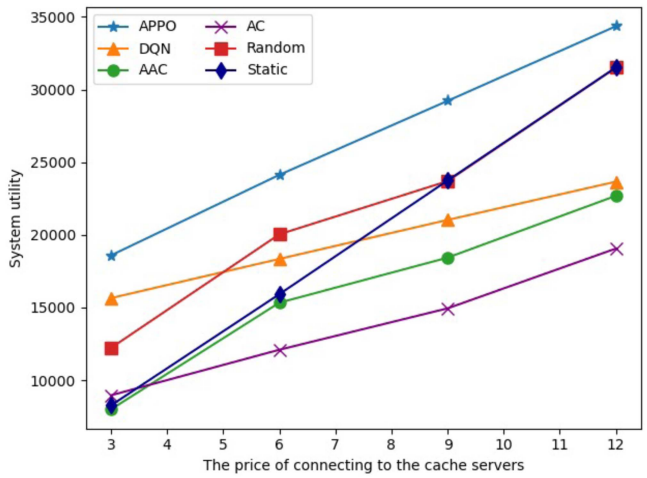

Fig. 12. The impact of the price of connecting to the cache servers on system utility under different algorithm.

APPO proposed in this paper can make better decision-making and thus get a higher average system utility. This is because as the number of tasks increases the environment becomes more complicated. However, this paper considers the impact of tasks on the future RPs in the dynamic RP model, and APPO algorithm can obtain RPs' states in a complex environment by adopting an adaptive selection policy.

Fig. 11 shows the effect of the unit price of the completed task on the system utility under different algorithms. Here we assume that the unit price for completing different tasks is the same. It can be seen from the figure that as the unit price increases, the system utility obtained by the APPO algorithm shows an upward trend. When the price of tasks is 1 unit/Mbps, the system utility obtained by APPO is not much different from that obtained by ACC and DQN. However, with the increase in unit price, other algorithms gradually widened the gap with APPO algorithm, and the maximum gap between system utility is about 50,000. Fig. 12 shows the impact of different cache prices on system utility. It can be seen that the advantages of APPO algorithm. As the caching price continues to rise, each algorithm has become a trend, and the results of APPO maintain the highest system utility.

From the figures, we can see that with the increase in computing and cache prices, traditional AC does not have a higher performance improvement than APPO. Task offloading has a high diversity. If only the current environment state is considered, the algorithms cannot make better global decisions. The agent does not consider the RP that contributes greatly to the system utility, so its performance is relatively poor. As can be seen from the resulting graph, the broken line of the APPO algorithm will be smoother and more stable, while the traditional allocation method has slight fluctuations. This shows to a certain extent that considering the contribution of resource pools to the system can improve the overall stability of the model.

VII. Conclusion

In this article, to make full use of the idle computing and caching resource in the CPN and evaluate the contribution of each pool participating in the learning progress, we propose an in-network pooling framework in B5G/6G era and the attention-based deep reinforcement learning algorithm. Wherein, the dynamic RP is first constructed to make full use of idle network resources, after which the jointly computing and caching problem is stated as the maximizing of long-term system utility, and lastly the problem is solved using the APPO. Combining the attention mechanism reveals the contribution of RPs to the system. The proposed algorithm effectively outperforms other existing algorithms, shown by the experimental results.

References

[1] F. Tariq, M. R. Khandaker, K.-K. Wong, M. A. Imran, M. Bennis, and M. Debbah, "A speculative study on 6G," *IEEE Wireless Commun.*, vol. 27, no. 4, pp. 118–125, Aug. 2020.

[2] Z. Zhou, X. Chen, E. Li, L. Zeng, K. Luo, and J. Zhang, "Edge intelligence: Paving the last mile of artificial intelligence with edge computing," *Proc. IEEE*, vol. 107, no. 8, pp. 1738–1762, Aug. 2019.

[3] J. Liu, K. Luo, Z. Zhou, and X. Chen, "ERP: Edge resource pooling for data stream mobile computing," *IEEE Internet Things J.*, vol. 6, no. 3, pp. 4355–4368, Jun. 2019.

[4] C. Tang, C. Zhu, X. Wei, W. Chen, and J. J. P. C. Rodrigues, "Rsu-empowered resource pooling for task scheduling in vehicular fog computing," in *Proc. Int. Wireless Commun. Mobile Comput.*, 2020, pp. 1758–1763.

[5] W. Y. B. Lim, J. S. Ng, Z. Xiong, D. Niyato, C. Miao, and D. I. Kim, "Dynamic edge association and resource allocation in self-organizing hierarchical federated learning networks," *IEEE J. Sel. Areas Commun.*, vol. 39, no. 12, pp. 3640–3653, Dec. 2021.

[6] S. Hu, Y.-C. Liang, Z. Xiong, and D. Niyato, "Blockchain and artificial intelligence for dynamic resource sharing in 6G and beyond," *IEEE Wireless Commun.*, vol. 28, no. 4, pp. 145–151, Aug. 2021.

[7] I. A. Elgendy, W.-Z. Zhang, H. He, B. B. Gupta, and A. A. Abd El-Latif, "Joint computation offloading and task caching for multi-user and multi-task mec systems: Reinforcement learning-based algorithms," *Wireless Netw.*, vol. 27, no. 3, pp. 2023–2038, 2021.

[8] C. Li, L. Zhu, W. Li, and Y. Luo, "Joint edge caching and dynamic service migration in SDN based mobile edge computing," *J. Netw. Comput. Appl.*, vol. 177, 2021, Art. no. 102966.

[9] S. Liu, C. Zheng, Y. Huang, and T. Q. S. Quek, "Distributed reinforcement learning for privacy-preserving dynamic edge caching," *IEEE J. Selected Areas Commun.*, vol. 40, no. 3, pp. 749–760, 2022.

[10] X. Wang, X. Ren, C. Qiu, Y. Cao, T. Taleb, and V. C. Leung, "Net-in-AI: A computing-power networking framework with adaptability, flexibility, and profitability for ubiquitous AI," *IEEE Netw.*, vol. 35, no. 1, pp. 280–288, Jan./Feb. 2021.
[11] N. Hu, Z. Tian, X. Du, and M. Guizani, "An energy-efficient in-network computing paradigm for 6G," *IEEE Trans. Green Commun. Netw.*, vol. 5, no. 4, pp. 1722–1733, Dec. 2021.
[12] X. Tang et al., "Computing power network: The architecture of convergence of computing and networking towards 6G requirement," *China Commun.*, vol. 18, no. 2, pp. 175–185, 2021.
[13] M. Katz, F. H. Fitzek, D. E. Lucani, and P. Seeling, "Mobile clouds as the building blocks of shareconomy: Sharing resources locally and widely," *IEEE Veh. Technol. Mag.*, vol. 9, no. 3, pp. 63–71, Sep. 2014.
[14] M. Afrin, J. Jin, A. Rahman, A. Rahman, J. Wan, and E. Hossain, "Resource allocation and service provisioning in multi-agent cloud robotics: A comprehensive survey," *IEEE Commun. Surveys Tuts.*, vol. 23, no. 2, pp. 842–870, Apr.–Jun. 2021.
[15] Y. Mansouri and M. A. Babar, "A review of edge computing: Features and resource virtualization," *J. Parallel Distrib. Comput.*, vol. 150, pp. 155–183, 2021.
[16] G. Qu, H. Wu, R. Li, and P. Jiao, "DMRO: A deep meta reinforcement learning-based task offloading framework for edge-cloud computing," *IEEE Trans. Netw. Service Manag.*, vol. 18, no. 3, pp. 3448–3459, Sep. 2021.
[17] C. Li, M. Song, C. Yu, and Y. Luo, "Mobility and marginal gain based content caching and placement for cooperative edge-cloud computing," *Inf. Sci.*, vol. 548, pp. 153–176, 2021.
[18] T. Yang, H. Feng, C. Yang, Y. Wang, J. Dong, and M. Xia, "Multivessel computation offloading in maritime mobile edge computing network," *IEEE Internet Things J.*, vol. 6, no. 3, pp. 4063–4073, Jun. 2019.
[19] J. Liu, J. Ren, Y. Zhang, X. Peng, Y. Zhang, and Y. Yang, "Efficient dependent task offloading for multiple applications in mec-cloud system," *IEEE Trans. Mobile Comput.*, vol. 22, no. 4, pp. 2147–2162, Apr. 2023.
[20] A. Samanta and J. Tang, "Dyme: Dynamic microservice scheduling in edge computing enabled IoT," *IEEE Internet Things J.*, vol. 7, no. 7, pp. 6164–6174, Jul. 2020.
[21] J. Feng, F. R. Yu, Q. Pei, J. Du, and L. Zhu, "Joint optimization of radio and computational resources allocation in blockchain-enabled mobile edge computing systems," *IEEE Trans. Wireless Commun.*, vol. 19, no. 6, pp. 4321–4334, Jun. 2020.
[22] Q. Zhang, L. Gui, F. Hou, J. Chen, S. Zhu, and F. Tian, "Dynamic task offloading and resource allocation for mobile-edge computing in dense cloud RAN," *IEEE Internet Things J.*, vol. 7, no. 4, pp. 3282–3299, Apr. 2020.
[23] S. Hu and G. Li, "Dynamic request scheduling optimization in mobile edge computing for IoT applications," *IEEE Internet Things J.*, vol. 7, no. 2, pp. 1426–1437, Feb. 2020.
[24] Y. Jie, C. Guo, K.-K. R. Choo, C. Z. Liu, and M. Li, "Game-theoretic resource allocation for fog-based industrial Internet of Things environment," *IEEE Internet Things J.*, vol. 7, no. 4, pp. 3041–3052, Apr. 2020.
[25] Z. Sun and M. R. Nakhai, "Distributed mirror-prox optimization for multi-access edge computing," *IEEE Trans. Commun.*, vol. 68, no. 5, pp. 3096–3106, May 2020.
[26] Y. Xu, G. Gui, H. Gacanin, and F. Adachi, "A survey on resource allocation for 5G heterogeneous networks: Current research, future trends, and challenges," *IEEE Commun. Surv. Tuts.*, vol. 23, no. 2, pp. 668–695, Apr.–Jun. 2021.
[27] R. Gu, Z. Yang, and Y. Ji, "Machine learning for intelligent optical networks: A comprehensive survey," *J. Netw. Comput. Appl.*, vol. 157, 2020, Art. no. 102576.
[28] S. Vimal, M. Khari, N. Dey, R. G. Crespo, and Y. Harold Robinson, "Enhanced resource allocation in mobile edge computing using reinforcement learning based Moaco algorithm for IIoT," *Comput. Commun.*, vol. 151, pp. 355–364, 2020.
[29] S. Gong, Y. Xie, J. Xu, D. Niyato, and Y.-C. Liang, "Deep reinforcement learning for backscatter-aided data offloading in mobile edge computing," *IEEE Netw.*, vol. 34, no. 5, pp. 106–113, Sep./Oct. 2020.
[30] W. Zhan et al., "Deep-reinforcement-learning-based offloading scheduling for vehicular edge computing," *IEEE Internet Things J.*, vol. 7, no. 6, pp. 5449–5465, Jun. 2020.
[31] Z. Zhou, K. Luo, and X. Chen, "Deep reinforcement learning for intelligent cloud resource management," in *Proc. IEEE Conf. Comput. Commun. Workshops*, 2021, pp. 1–6.
[32] Y. Ren, X. Chen, S. Guo, S. Guo, and A. Xiong, "Blockchain-based VEC network trust management: A DRL algorithm for vehicular service offloading and migration," *IEEE Trans. Veh. Technol.*, vol. 70, no. 8, pp. 8148–8160, Aug. 2021.
[33] Z. Xie, R. Wu, M. Hu, and H. Tian, "Blockchain-enabled computing resource trading: A deep reinforcement learning approach," in *Proc. IEEE Wireless Commun. Netw. Conf.*, 2020, pp. 1–8.
[34] S. Luo, X. Chen, Q. Wu, Z. Zhou, and S. Yu, "HFEL: Joint edge association and resource allocation for cost-efficient hierarchical federated edge learning," *IEEE Trans. Wireless Commun.*, vol. 19, no. 10, pp. 6535–6548, Oct. 2020.
[35] Z. Zhou, S. Yang, L. Pu, and S. Yu, "CEFL: Online admission control, data scheduling, and accuracy tuning for cost-efficient federated learning across edge nodes," *IEEE Internet Things J.*, vol. 7, no. 10, pp. 9341–9356, Oct. 2020.
[36] J. S. Ng et al., "A hierarchical incentive design toward motivating participation in coded federated learning," *IEEE J. Sel. Areas Commun.*, vol. 40, no. 1, pp. 359–375, Jan. 2022.
[37] W. Y. B. Lim et al., "Decentralized edge intelligence: A dynamic resource allocation framework for hierarchical federated learning," *IEEE Trans. Parallel Distrib. Syst.*, vol. 33, no. 3, pp. 536–550, Mar. 2022.
[38] R. Yao, G. Lin, S. Xia, J. Zhao, and Y. Zhou, "Video object segmentation and tracking: A survey," *ACM Trans. Intell. Syst. Technol.*, vol. 11, no. 4, pp. 1–47, May 2020.
[39] A. Shakarami, M. Ghobaei-Arani, and A. Shahidinejad, "A survey on the computation offloading approaches in mobile edge computing: A. machine learning-based perspective," *Computer Netw.*, 2020, Art. no. 107496.
[40] D. Zhang et al., "Near-optimal and truthful online auction for computation offloading in green edge-computing systems," *IEEE Trans. Mobile Comput.*, vol. 19, no. 4, pp. 880–893, Apr. 2020.
[41] Y. He, C. Liang, F. R. Yu, and Z. Han, "Trust-based social networks with computing, caching and communications: A deep reinforcement learning approach," *IEEE Trans. Netw. Sci. Eng.*, vol. 7, no. 1, pp. 66–79, Jan.–Mar. 2020.
[42] Y. He, N. Zhao, and H. Yin, "Integrated networking, caching, and computing for connected vehicles: A deep reinforcement learning approach," *IEEE Trans. Veh. Technol.*, vol. 67, no. 1, pp. 44–55, Jan. 2018.
[43] F. Pizzocaro, D. Torreggiani, and G. Gilardi, "Inhibition of apple polyphenoloxidase (PPO) by ascorbic acid, citric acid and sodium chloride," *J. Food Process. Preservation*, vol. 17, no. 1, pp. 21–30, 1993.
[44] S. Iqbal and F. Sha, "Actor-attention-critic for multi-agent reinforcement learning," in *Proc. 36th Int. Conf. Mach. Learn.* (Proceedings of Machine Learning Series), K. Research Chaudhuri and R. Salakhutdinov, Eds., 2019, vol. 97, pp. 2961–2970.
[45] F. Fu, Z. Zhang, F. R. Yu, and Q. Yan, "An actor-critic reinforcement learning-based resource management in mobile edge computing systems," *Int. J. Mach. Learn. Cybern.*, vol. 11, no. 8, pp. 1875–1889, 2020.
[46] Y. Zhao, R. Li, C. Wang, X. Wang, and V. C. Leung, "Neighboring-aware caching in heterogeneous edge networks by actor-attention-critic learning," in *Proc. IEEE Int. Conf. Commun.*, 2021, pp. 1–6.
[47] C. Qiu, X. Wang, H. Yao, J. Du, F. R. Yu, and S. Guo, "Networking integrated cloud-edge-end in IoT: A blockchain-assisted collective q-learning approach," *IEEE Internet Things J.*, vol. 8, no. 16, pp. 12694–12704, Aug. 2021.
[48] T. D. Braun, H. J. Siegel, A. A. Maciejewski, and Y. Hong, "Static resource allocation for heterogeneous computing environments with tasks having dependencies, priorities, deadlines, and multiple versions," *J. Parallel Distrib. Comput.*, vol. 68, no. 11, pp. 1504–1516, 2008.
[49] M. Chen and Y. Hao, "Task offloading for mobile edge computing in software defined ultra-dense network," *IEEE J. Sel. Areas Commun.*, vol. 36, no. 3, pp. 587–597, Mar. 2018.

Zheng Di (Student Member, IEEE) received the B.S. degree in software engineering from Tianjin Normal University, Tianjin, China, in 2018. She is currently working toward the M.S. degree with the School of Computer Science and Technology, College of Intelligence and Computing, Tianjin University. Her research interests include edge computing and computing power network.

Tao Luo received M.S. degree from the School of Precision Instrument and Opto-Electronics in 2006, and the Ph.D. degree from the School of Electronic Information Engineering, Tianjin University, Tianjin, China, in 2009. He is currently an Associate Professor with the College of Intelligence and Computing, Tianjin University. His research interests include machine learning, edge computing, and blockchain.

Chao Qiu (Member, IEEE) received the B.S. degree from China Agricultural University, Beijing, China, in 2013 in communication engineering. She is currently working toward the Ph.D. degree with the School of Information and Communication Engineering, Beijing University of Posts and Telecommunications, Beijing, China. From September 2017 to September 2018, she visited Carleton University, Ottawa, ON, Canada, as a Visiting Scholar. Her research interests include machine learning, software defined networking, and blockchain.

Cheng Zhang received the Ph.D degree from Tianjin University, Tianjin, China, in computer application technology in 2021. From September 2018 to September 2019, he visited Beijing Institute of Technology, as a Visiting Scholar. He is currently a Lecturer with the College of Science and Technology, Tianjin University of Financial and Economics, Tianjin, China, and a Researcher with the Haihe Laboratory of Information Technology Application Innovation. His research interests include deep learning and model interpretability.

Zhutao Liu (Student Member, IEEE) received the B.S. degree from the School of Software, College of Intelligence and Computing, Tianjin University, Tianjin, China, in 2021, where he is currently working toward the M.S. degree. His research interests include edge caching, distributed federated learning optimization, blockchain, and deep learning.

Xiaofei Wang (Senior Member, IEEE) received the B.S. degree from the Huazhong University of Science and Technology, Wuhan, China, and the M.S. and Ph.D. degrees from Seoul National University, Seoul, South Korea, in 2005, 2008 and 2013 respectively. He was a Postdoctoral Fellow with The University of British Columbia, Vancouver, BC, Canada, from 2014 to 2016. He is currently a Professor with the College of Intelligence and Computing, Tianjin University, Tianjin, China. Focusing on the research of edge computing, edge intelligence, and edge systems, he has authored or coauthored more than 160 technical papers in IEEE JSAC, TCC, ToN, TWC, IoTJ, COMST, TMM, INFOCOM, ICDCS and so on. He has was the recipient of the Best Paper awards of IEEE ICC, ICPADS, and in 2017, the IEEE ComSoc Fred W. Ellersick Prize, and in 2022, the IEEE ComSoc Asia-Pacific Outstanding Paper Award.

Jing Jiang (Member, IEEE) received the M.Sc. degree from Xi Dian University, Xi'an, China, in 2005, and the Ph.D. degree in information and communication engineering from North Western Polytechnic University, Xi'an, China, in 2009. She was the Leader of 3GPP LTE MIMO Project with ZTE Corporation, China, from 2006 to 2013. She is currently a Professor with the Shaanxi Key Laboratory of Information Communication Network and Security, Xi'an University of Posts and Telecommunications, Xi'an, China. Her research interests include edge caching, blockchain technology and Artificial Intelligence based wireless communication.